实验 04 | EX：AI Harness 配置及 107 本科生算力平台介绍

Windows 本地协作 · 107 平台 CPU/GPU 实践

本次实验介绍 AI Harness 工具与 107 本科生算力平台，供后续实验和大作业参考。AI Harness 可以配合模型读取文件、修改代码和运行程序；107 平台提供远程 CPU/GPU 资源，用于运行计算与训练任务。

第一部分以 Windows 上的 Codex + DeepSeek 为例，介绍模型、API 与 Harness 的区别及基本配置，再使用实验 02 的决策树代码练习 AI 辅助编程。第二部分介绍 107 平台的使用，先提交一个 CPU 小程序，再运行 CIFAR-10 + ResNet-18 图像分类训练。

本实验供自主学习，不要求提交，也不计入评分。请先完整解压配套材料，再结合讲义操作；已有使用经验的同学可以直接查阅需要的章节。项目迁移、配置管理和 DSH 等补充内容见附录。

目录

第一部分：AI Harness 工具配置与协作 2
1.1 模型、API 与工具调用 2
1.2 Codex 安装与 DeepSeek API 配置 6
1.3 用决策树代码练习 AI 辅助编程 10
第二部分：107 本科生算力平台实践 13
2.1 平台入门与 CPU 任务 13
2.2 CIFAR-10 图像分类的 GPU 短训练 15
2.3 结果下载与问题排查 19
附录 A：图像分类项目迁移与较大训练（选读） 22
附录 B：配置管理与工具扩展（选读） 24
附录 C：DSH 介绍与配置（选读） 25
附录 D：用 Git 保存版本，降低试错成本（选读） 28

第一部分：AI Harness 工具配置与协作

本部分使用 Codex 连接 DeepSeek API，并用实验 02 的决策树代码练习 AI 辅助编程。练习沿用实验 01/02 的 Python 环境。

1.1 模型、API 与工具调用

1.1.1 从网页聊天到文件操作

很多同学已经用过网页版 AI，向它提问或请它生成文字。下面用整理笔记的例子说明，AI 接入工具后可以多做哪些事情：

示例文本

```
今天看过 Python 的列表、字典和函数。
函数参数还不熟，需要再看一个例子。
下一次想读决策树程序的 fit 和 predict 方法。
```

聊天问答

在聊天窗口里问“函数参数是什么意思”，AI 可以给出解释和例子。你阅读回答，再针对不明白的地方追问。

请 AI 整理文字

把这段笔记贴进聊天，要求“按已学内容和待复习内容整理成 Markdown，保留原有信息”。AI 给出文字后，你把它复制到编辑器并保存。需要修改时，再把相关内容贴回聊天窗口。

让 Agent 操作文件

如果客户端提供文件读写工具，AI 就可以直接整理文件。假设笔记保存在练习目录的 `notes.txt` 中，可以这样要求：

示例文本

```
请读取当前练习目录中的 notes.txt，
按已学内容和待复习内容整理成 study-notes.md。
保留原有信息，操作范围仅限这两个文件。
完成后读取生成文件，核对内容，并说明实际结果。
```

工具和权限允许时，Agent 可以读取笔记、写出整理后的文件，再读取文件检查。你也可以继续要求“把待复习内容改成简短列表”，让它直接修改已有文件，省去反复复制粘贴。

具体能做哪些操作，取决于模型能力、客户端提供的工具和权限设置。网页或桌面只是界面形式，一些网页产品也能调用工具。使用时可以查看工具记录，确认它是否真的读取、修改了文件。

1.1.2 Transformer 的基本原理

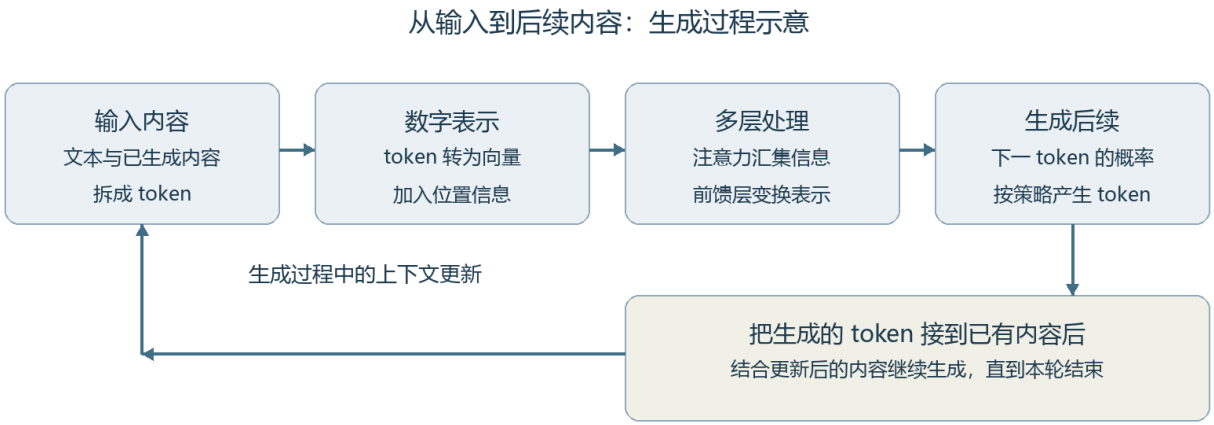
Transformer 是许多语言模型采用的神经网络结构。下面的图示概括了生成文本时的基本过程，不展开矩阵运算；不同模型的具体结构会有差异。

我们的正课中也会进行更加详细的 Transformer 介绍，这里只抽象性地说明它怎样生成文本，以及模型、API 与工具调用的关系。

同属 Transformer，模型的结构与用途也会不同。几个经典例子如下：

模型示例	结构	常见用途
BERT	编码器：结合整段输入得到表示	文本分类、信息提取
GPT 类模型	解码器：根据已有内容逐个生成 token	文本续写与生成
T5	编码器与解码器：读取输入，再生成输出	翻译、摘要等文本转换

下图以逐个生成 token 的过程为例说明。



1. **将文本转成数字。** 文本先拆成 token（词元），每个 token 再表示为向量，并加入顺序或位置信息。Token 怎样划分取决于模型，不一定对应一个字或一个完整词。
2. **利用上下文。** 注意力机制根据不同位置的表示计算权重，将信息加权汇总。例如“把笔记保存成文件，再修改它”中的“它”，需要结合前文理解。这个句子只是上下文关系的示例。模型还会通过前馈层等部件变换表示，并重复多层处理。
3. **生成下一个 token。** 模型计算下一 token 的概率，按生成策略选择一个，接到已有内容后继续计算。重复这个过程，就能生成回答或代码。Transformer 原论文介绍了注意力等部件。

训练通过数据调整模型参数；使用训练好的模型处理输入，叫作推理。日常聊天时补充要求，通常改变的是模型下一次收到的上下文，不会当场重新训练模型。客户端也可能保存会话或资料，具体取决于软件功能。

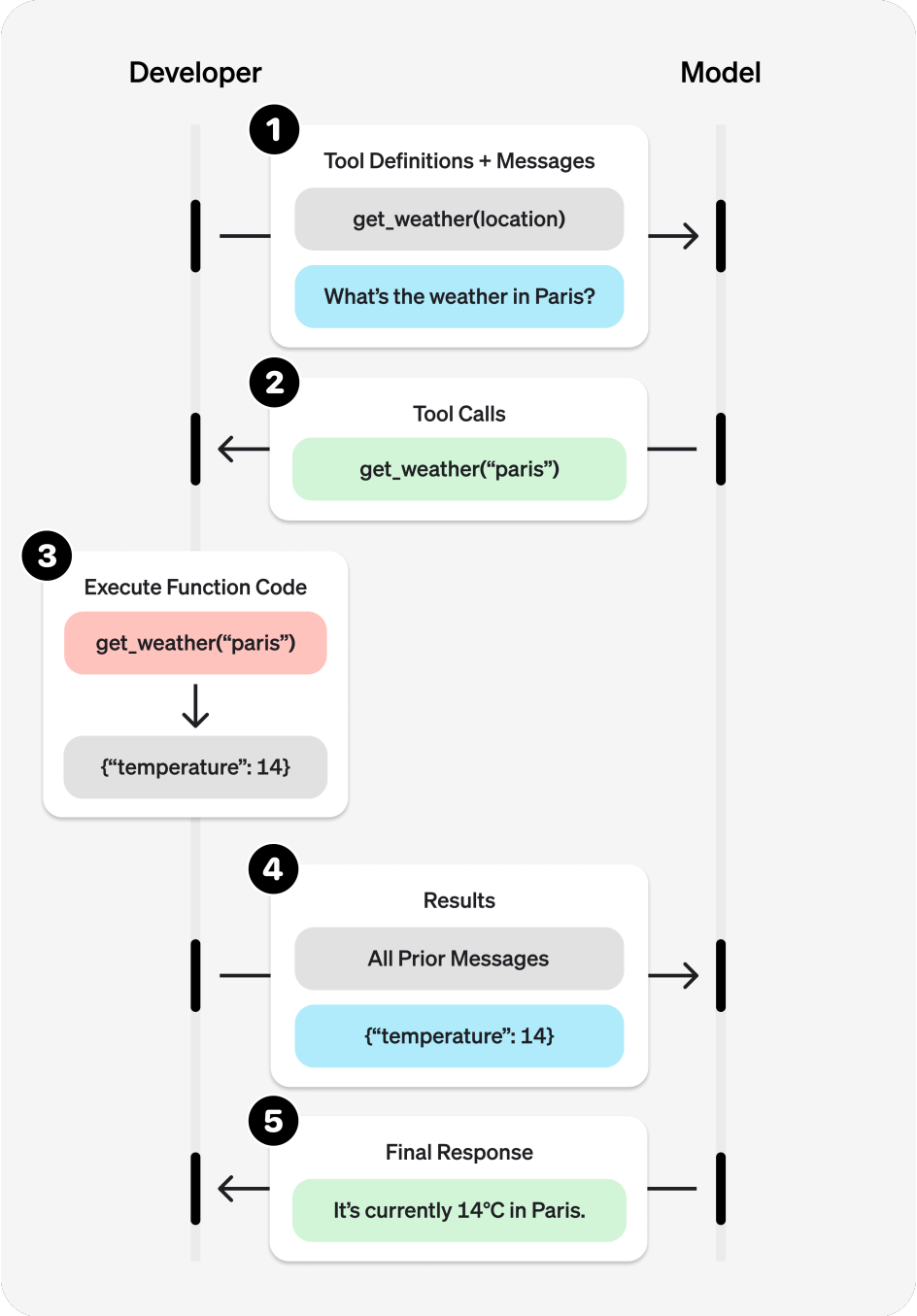
大模型可能生成听起来合理、实际上错误或不存在的信息，这类现象称为大模型幻觉（hallucination）。例如，回答可能编造参考文献，生成的代码也可能调用不存在的函数。回答的语气很肯定，并不能说明内容可靠。

然而，在实际项目中，我们的工作往往不会停在对话里。信息需要联网查证，爬取的数据需要运行脚本清洗，检查后的代码也可能需要通过 Git[1] 推送（push）到远端，供协作者审阅与合并（merge）。这些动作都要由实际工具完成；下一节介绍模型怎样请求调用工具。

[1]Git 是记录代码版本和协作改动的工具。`commit` 保存本地提交，`push` 将提交上传到远端仓库，`merge` 合并分支改动；推送成功不代表已合并。本地保存与恢复版本见附录 D，远端操作详见 Git push 与 Git merge 文档。

1.1.3 模型如何调用工具

支持工具调用的模型可以根据工具说明生成调用请求。客户端执行获准的请求，将结果返回给模型；模型据此决定下一步操作或生成回答。工具调用说明介绍了这一过程。



图源：OpenAI 工具调用文档。左侧 Developer 表示运行工具的应用程序，本实验中对应本地客户端；右侧 Model 表示模型服务。应用先发送工具说明与问题，模型返回调用请求；应用实际查询天气后，把结果送回模型，模型再生成回答。第 3 步的查询由应用执行，不发生在模型内部。

以刚才的笔记整理为例，执行过程如下。具体工具名称和调用格式随软件变化：

示例文本

学生给出目标与范围
→ 模型请求读取 `notes.txt`
→ 程序读取文件，把内容返回模型
→ 模型生成整理内容并请求写入 `study-notes.md`
→ 程序写入并返回结果
→ 模型结合结果继续检查或回复
→ 学生打开文件，检查并提出修订

模型收到工具结果后，可以继续请求读取生成文件、检查内容，或按你的新要求修改。写入成功不代表内容正确，最终仍需打开文件核对。

模型、API、Harness 与 Agent

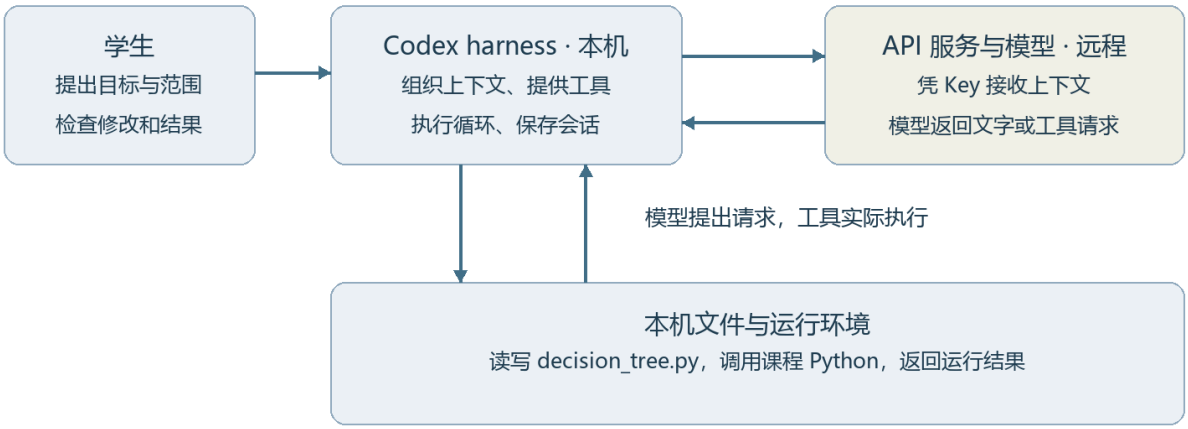
模型根据上下文生成回答、代码或工具调用请求。第一部分使用 DeepSeek 提供的模型。

API 是程序之间通信的接口。模型服务通过 API 接收请求并返回结果；服务地址决定请求发到哪里，API Key 用来识别调用者及其权限。不同平台提供同名模型时，速度、额度和价格也可能不同。

Harness 是运行 Agent 的软件框架，负责整理上下文、提供工具、执行获准的操作，以及将结果返回模型。Codex 属于这类工具，本实验使用它的桌面客户端。

Agent 在本文中指模型与工具结合后，能根据目标连续执行步骤的助手。模型提出“读取文件”的请求，Harness 提供的工具执行读取，并受当前权限限制。

一次 agent 协作涉及的层次



在本实验的配置中，模型推理发生在 API 服务端，文件读写和 Python 程序在你的电脑上执行。因此不需要在本机部署 DeepSeek 模型，决策树练习只需已有的课程 Python 环境。发送给模型的上下文可能包含代码和工具输出，练习使用公开课程材料，不放入个人敏感资料。

Codex 这个名称也用于模型产品。本文所说的 Codex 指客户端和运行框架；连接 DeepSeek 后，使用哪个模型由 API 配置决定，要看服务地址和模型 ID，不能只看回答中的自我介绍。

配置其他工具时，也要分别确认客户端、API 服务、模型 ID 和工具权限。下节以 Codex + DeepSeek 为例说明。

1.2 Codex 安装与 DeepSeek API 配置

以下使用 DeepSeek 官方 API。已有学校 API Key 的同学可先看第 1.2.2 节，确认服务地址和客户端支持的接口。

1.2.1 API 费用与网络条件

API 怎样计费

本实验使用按量计费的 DeepSeek 官方 API。客户端或配置管理器免费，不代表 API 调用免费；桌面账号的订阅额度也不能当作 DeepSeek API 余额。使用学校服务时，查看学校项目的授权额度。

前面介绍的 token 也是 API 常用的计费单位，不能按固定比例换算成中文字符。费用通常分别统计：

- **输入**：你发出的要求，以及程序随请求传入的指令、代码、历史消息和工具结果。
- **缓存输入**：服务端复用了此前处理过的输入前缀，可能采用较低单价；仍可能计费。
- **输出**：模型生成的内容；推理内容的计费口径也应查看服务商说明。

一次 Agent 任务可能调用模型多次。每次读文件、运行程序或修改代码后，工具结果都可能进入下一次请求。所以在聊天框中只发一条消息，也可能产生多次调用的费用。

下面是 **2026-10-12 核对的 `deepseek-flash` 人民币价格示例**，单位为元／每百万 token。充值前再看官方中文价格页，以账户实际结算为准。

类型	低谷价格	高峰价格
输入，缓存命中	0.02	0.04
输入，缓存未命中	1.00	2.00
输出	4.00	8.00

例如一次请求在低谷时段消耗 10,000 个未缓存输入 token、20,000 个缓存输入 token 和 3,000 个输出 token：

示例文本

```
费用 = (10000 × 1.00 + 20000 × 0.02 + 3000 × 4.00) / 1000000
     = 0.0224 元
```

整次练习的费用取决于实际调用次数和 token 用量。可以先做第 1.3 章的 T1 小任务，再到平台查看用量和余额。遇到反复重试时先停止，查清错误后再继续。

国内网络下的安装与调用

DeepSeek 官方提供了 Codex 配置脚本，并支持 Codex 使用的 Responses API。实际使用还需要下载安装客户端、取得 Key、运行脚本，再测试模型和工具调用。安装源与 API 地址是不同的网络入口，一个能访问不代表另一个也能访问。

这套方案尚未在干净、无代理的 Windows 环境中完成全流程预演。安装或启动受限时，可参考学校 API 与其他客户端；DSH 的试用条件见附录 C。排错时逐项检查，不要同时反复改动多套配置。

1.2.2 学校 API 与其他客户端

申请学校 API

打开中国科大大模型公共服务平台，按申请流程通过统一身份认证登录，创建用途明确的项目，再申请 API Key。免费资源的可用模型、额度与有效期以实际授权为准。

学校 Key 需要配合学校网关使用，不能填入下文固定连接 DeepSeek 官网的脚本。模型 ID 从项目的可用模型列表复制。具体客户端配置可参考学校的图文入门指南。

学校服务由多人共享，回复速度可能不及官方或较大平台的 API，尤其在使用人数较多时。建议先用短任务试用，观察等待时间和生成速度[2]，再决定是否运行较长任务。

其他客户端入口

情况	可以尝试的路径
Codex 桌面下载或启动不顺	可看 CLI 补充入口；DSH 桌面版的安装与配置见附录 C
有学校 Key，希望图形化操作	参考学校指南配置 WorkBuddy；DSH 的介绍与配置见附录 C
主要处理办公文件	参考学校指南中的 WorkBuddy 配置方式
已熟悉终端，桌面入口受限	使用 Codex CLI 作为补充入口

学校指南中的社区 DSH Desktop 与 DeepSeek 官方桌面发行版不是同一个安装包，设置界面和数据目录可能不同。跟做时以所选发行版的说明为准。

学校的 Chat Completions 基础地址通常为 `https://api.llm.ustc.edu.cn/v1`；WorkBuddy 示例要求填写完整接口地址 `https://api.llm.ustc.edu.cn/v1/chat/completions`。Codex 使用 Responses 接口，接学校服务前需确认是否已开通，以及哪些模型可用。其他客户端也可以使用本实验的决策树练习。

1.2.3 安装 Codex 并连接 DeepSeek

安装和配置可以结合参考演示视频（B 站）操作。视频同样使用 DeepSeek 官方脚本。界面和模型名称可能随版本变化，填写时参考当前官方说明。

步骤 1：准备客户端和 API Key

1. 从官方 Windows 桌面说明进入下载与安装流程。新版本可能以 ChatGPT 桌面应用承载 Codex，界面名称以官方下载页为准。
2. 安装后启动一次。DeepSeek 脚本需要已有的用户配置目录；默认位置是 `%USERPROFILE%\.codex`。
3. 打开 DeepSeek API 平台，申请自己的 API Key，并查看当前余额与计费规则。已有学校 API Key 的同学先看学校 API 说明，学校 Key 与 DeepSeek 官方 Key 对应不同服务地址。

首次启动可能出现登录或模式选择。若找不到自定义 API 入口，保留界面提示，参考其他客户端入口或 CLI 说明。此时还未发送模型请求，先处理客户端启动问题。

[2] TPS（tokens per second）指每秒生成的 token 数。首个 token 出现前的等待时间，以及整次任务的耗时，也会影响使用体验。

API Key 是调用凭据。把它保存在可信客户端的密钥设置中，或按官方接入步骤处理。不要放进聊天提示词、公开代码和共享截图。泄露后应在对应平台撤销旧 Key，再创建新的 Key。

步骤 2：运行官方配置脚本

先完全退出桌面应用[3]，再打开普通 PowerShell。以下命令来自 DeepSeek 官方 Codex 接入文档，会下载并执行配置脚本[4]：

PowerShell · Windows

```
irm https://cdn.deepseek.com/api-docs/codex-deepseek-setup-en.ps1 | iex
```

按当前菜单选择 `1`，使用 `deepseek-flash`，然后填写自己的 DeepSeek API Key。当前脚本的输入框不遮住密钥字符，操作时避免屏幕共享和录屏。

脚本会备份旧配置，写入模型信息和连接设置，然后提示重启应用。它只配置已有客户端，不测试 API Key 是否可用；还需要完成步骤 3 的问答测试。

已有自定义配置时，先查看 Windows 脚本源码中的变更说明。脚本会处理部分冲突设置，包括 profile 和指令覆盖项。恢复选项还原的是配置脚本运行前的状态，运行后新增的设置需另行保存。

步骤 3：先做文字问答，再做工具操作

重新打开桌面应用，确认当前模型显示与 DeepSeek 配置一致。新建会话并发送：

示例文本

```
请用三句话解释信息熵。这次只回答文字，不读取或修改文件。
```

收到回答后，到 DeepSeek 平台查看调用用量。然后按决策树练习打开项目副本，测试文件读取和 Python 执行；这些操作还需要客户端的工具和权限支持。

如果模型选择或界面名称与教程不同，优先检查实际提供方和模型 ID；完全退出应用后再打开，关闭窗口可能仍让程序留在后台。

步骤 4：了解配置文件

关闭客户端后，在资源管理器打开 `%USERPROFILE%\.codex`[5]，查看下面几个配置项。配置可能含有明文 Key，求助时不要发送未经遮挡的全文。

[3]按 `Ctrl+Shift+Esc` 打开任务管理器，在“进程”中查找 Codex 或 ChatGPT。保存当前文件后，若客户端仍在后台运行，可选中对应应用并点击“结束任务”。

[4]`irm` 是 `Invoke-RestMethod` 的简写，`iex` 是 `Invoke-Expression` 的简写。这行命令从指定地址取得脚本，再执行它。

[5]按 `Win+E` 打开资源管理器，将完整路径粘贴到地址栏并按回车。`%USERPROFILE%` 表示当前 Windows 用户目录，例如 `C:\Users\你的用户名`。

信息	本教程的含义
`base_url`	DeepSeek 官方服务入口，当前脚本固定为 `https://api.deepseek.com/`
`wire_api`	请求协议，当前使用 `responses`
API Key	当前脚本以 `experimental_bearer_token` 写入本机配置；应遮住它
`model`	此次选择的模型 ID，主线为 `deepseek-flash`

`config.toml` 保存连接和运行设置；`models.json` 提供上下文长度、推理档位和工具格式等模型信息。更换模型时，这些设置也可能需要调整，不能只改模型名称。

“OpenAI 兼容”描述的是接口格式；实际模型提供方由服务地址和权限决定。Chat Completions 与 Responses 是不同接口，仅写“兼容 OpenAI”仍不足以判断 Codex 能否接入。Codex 配置说明介绍了这些字段。

1.2.4 安装与调用问题速查

现象	先检查什么
脚本提示配置目录不存在	客户端是否启动过；是否用了自定义配置目录
下载脚本失败	官方地址是否能打开；网络、TLS 或学校网络策略；先保留完整错误
401 / 403	Key 是否属于该服务；权限、有效期、余额与平台错误说明
404 / 参数不兼容	协议与地址是否匹配；是否给要求基础地址的字段填了完整接口
配置后仍显示旧模型	是否彻底退出应用再打开；当前会话是否加载了新配置
能回答文字，但不能改文件	工作目录、文件权限、工具调用兼容性与授权提示
agent 的 Python 命令失败	它使用的解释器路径是否与课程终端一致
初始程序输出 None	T1–T7 尚未补全；这是初始骨架的预期行为
回答很慢或反复重试	区分等待、限流与错误；停止重复请求后查看服务状态和用量

求助时给出客户端版本、服务提供方、模型 ID、操作步骤和完整错误；遮住 API Key。描述“问答成功、读取成功、运行失败”比只说“不能用”更容易定位。

多套配置管理和 CLI 使用见附录 B。

1.3 用决策树代码练习 AI 辅助编程

配套代码是实验 02 的初始骨架。A 阶段只补全 T1 信息熵，B 阶段再完成 T2–T7。使用独立副本，保留自己原先的实验 02 代码。

1.3.1 练习文件与 Python 环境

完整解压实验 04 材料包，将其中的 `code/decision-tree-practice/` 复制到自己的练习位置，例如 `文档\agent-lab2`。保留下列文件：

示例文本

```
agent-lab2/
  AGENTS.md
  README.md
  decision_tree.py
  checks.py
```

在 Codex 中选择这个文件夹作为工作目录，再新建会话。需要重新练习时，从材料包复制一份新的骨架即可。

打开课程终端，进入 `agent-lab2/`，确认能看到 `decision_tree.py` 和 `checks.py`。激活实验 01 创建的 `ustc-ai-experiments-26fall-B` 环境，查看 Python 版本和解释器路径，再运行程序：

PowerShell · Windows

```
conda activate ustc-ai-experiments-26fall-B
python --version
python -c "import sys; print(sys.executable)"
python decision_tree.py
```

版本应为 `3.11.x`，解释器应位于课程环境中。普通 PowerShell 找不到 `conda` 时，改用已有的 Miniforge Prompt。把打印出的 `python.exe` 完整路径告诉 Agent，因为桌面应用执行命令时可能没有继承这个终端的环境。

初始代码的 T1–T7 方法返回 `None`。演示出现 `None` 是占位结果，按步骤补全后会改变。程序只使用 Python 标准库，无需为这个练习安装 NumPy、scikit-learn 或额外的决策树库。

先给 agent 一个读取任务：

示例文本

```
请阅读 README.md、AGENTS.md 和 decision_tree.py。
说明数据怎么进入程序、训练和预测分别调用哪些方法，
并列出 T1–T7 的依赖关系。先只解释，不修改文件。
```

展开工具记录，查看读取的文件路径和返回内容，确认它打开的是这份练习副本。

1.3.2 A 阶段：补全 T1 信息熵

这次只补全 `_entropy`。你可以先手算两个结果：`[0, 1]` 的信息熵为 1，`[1, 1]` 的信息熵为 0。

把自己的解释器路径一并提供给 agent，然后发送：

示例文本

```
请只补全 decision_tree.py 的 T1：_entropy。
先简要解释思路，再修改这个方法，保持其余 TODO 不变。
遵守空列表返回 0.0 的约定，并跳过概率为 0 的项。
使用我指定的课程 Python 运行 python checks.py --stage a。
最后说明改了什么、实际运行了什么、哪些任务仍未完成。
```

执行命令或修改文件时，客户端可能请求授权。看清命令、目标文件夹和预期动作，再允许它执行。项目里的 `AGENTS.md` 能说明约定；实际访问控制仍由客户端权限和操作系统决定。



完成后自己检查：

1. 打开修改前后对比，确认改动集中在 T1。
2. 在自己的课程终端运行 `python checks.py --stage a`，预期为 `5/5 passed`。
3. 再运行 `python decision_tree.py`。信息熵应有数值，其余待补方法仍可能输出 `None`。
4. 追问：“为什么纯标签的信息熵是 0？为什么不能计算 `log2(0)`？”

自查脚本检查空输入、两种纯标签、均衡标签和不均衡标签。请看实际运行输出，确认各项通过。

如果 agent 顺便完成了其他 TODO，可以要求：“这次范围仅限 T1，请展示额外改动，并撤回那些改动。”修改前已有的内容要先区分，避免把自己的代码一起撤回。

1.3.3 B 阶段：完成 T2–T7

已有的 T1 保留。把任务扩展到完整实现，先让 agent 制定步骤：

示例文本

```
请基于当前 T1 实现，完成 T2–T7。
先按方法依赖给出简短计划，然后分阶段实现并运行验证。
保持原始特征列编号，遵守平票、零增益继续划分和未知分支回退规则。
不要修改内置数据、方法签名、题目约定或 checks.py。
使用我指定的课程 Python，最后运行 checks.py --stage b
和 decision_tree.py。汇报实际结果；失败时说明原因并继续修正。
```

可以按 T2–T4、T5、T6–T7 分段查看改动，确认各个方法怎样配合，并留意运行中出现的问题。

B 阶段自己运行：

PowerShell • Windows

```
python checks.py --stage b
python decision_tree.py
```

全部通过时应显示 `16/16 passed`。检查包括信息增益加权、增益并列顺序、递归终止条件、零增益的 XOR 小例子、原始列编号、未知分支和批量预测顺序。完整西瓜数据应选择“纹理”（列编号 3）作为根节点，信息增益约为 `0.380592`。

遇到失败时，把**失败项目、完整错误和当前改动**交给 agent，例如：

示例文本

```
自查中的未知分支项目失败，预期返回当前节点多数类。
请先读取相关实现并解释失败原因，再做最小修正。
修正后重新运行该项目和完整 B 阶段检查，保持 checks.py 不变。
```

完成后查看文件差异，确认内置数据和 `checks.py` 未被改动。对不明白的实现，可以让 Agent 用一个小例子解释，再对照代码检查。

自查通过只说明这些样例符合约定。本例在同一批 17 条记录上训练并回代预测，结果不能代表独立测试集上的泛化表现。

1.3.4 任务描述与 AGENTS.md

描述任务

上面的请求都说明了要读哪些文件、修改哪里，以及怎样检查。例如“只补全 T1，用指定 Python 运行 A 阶段自查”。范围越清楚，越容易发现额外改动；不熟悉代码时，可以先让 Agent 解释，再决定要它修改什么。

保存项目约定

练习中的 `AGENTS.md` 保存了先读说明、采用课程环境、遵守实验约定和运行自查等重复要求。

`AGENTS.md` 是给助手阅读的说明文件，不负责限制操作权限。约定尽量写具体，例如“保持内置数据不变”“运行 checks.py”，便于检查是否遵守。

了解读取位置与覆盖关系，可看 AGENTS.md 官方说明。新手先在练习根目录用一份即可。

继续或新建会话

同一个任务还在调试时，可以继续原会话。换了任务或会话过长时，新建会话并说明文件位置、已完成内容、最近的运行结果和接下来要做什么。

例如：“T1–T7 已完成，自查通过；请先读 decision_tree.py，解释未知分支怎样处理。”需要保留的成果应保存为文件，便于以后继续使用。

第二部分：107 本科生算力平台实践

107 本科生算力平台是学校面向本科教学与实践提供的计算服务。通过平台网页，可以管理项目文件、申请 CPU/GPU 资源、提交计算任务，并查看日志与下载结果。没有独立显卡的电脑也可以通过浏览器使用平台计算资源。

平台任务在远程 Linux 环境中执行。本地 Agent 可以帮助修改项目和脚本，上传、提交和下载由你在平台完成。本部分先运行 CPU 小程序，再尝试 GPU 图像分类训练；增加训练量的练习见附录 A。

2.1 平台入门与 CPU 任务

2.1.1 何时使用算力平台

决策树或小型表格分类练习通常可以在本地 CPU 上完成。训练图像模型、重复多组实验，或本机内存、显存不足时，可以考虑把任务放到平台，使用学校提供的 CPU/GPU 资源。

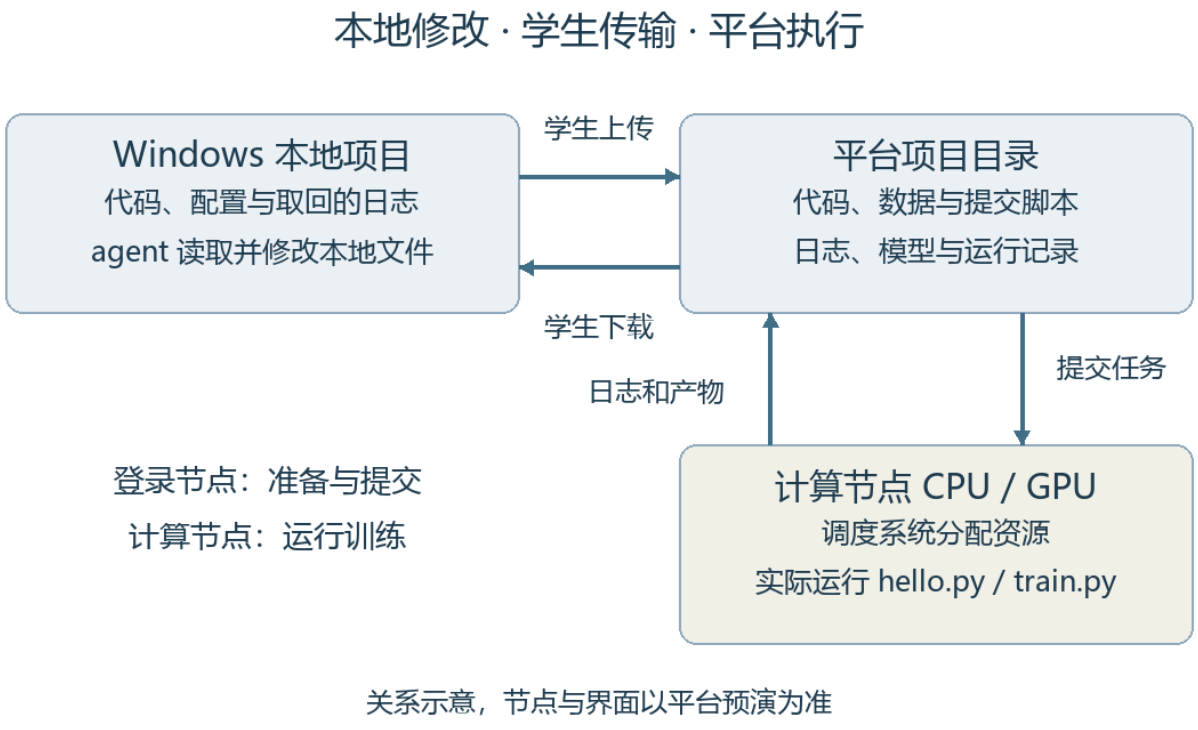
平台的 GPU 选项包括 RTX 5090 和 A100，实际可申请的型号与数量以当前账号和节点状态为准。短测试可以先申请单卡和较小 CPU/内存配置；增加训练量后，根据显存占用、数据加载速度与预计耗时调整资源。平台资源说明

CPU 处理通用运算和数据读取，GPU 适合神经网络中的大量张量计算。程序需要将模型和数据放到 GPU 上，申请 GPU 不会自动加速所有 Python 代码。CPU 使用的主内存与 GPU 显存也要分开看，申请更多主内存不能解决显存不足。

长时间 CPU 计算、数值模拟和批量实验也可以使用平台。使用前要考虑环境配置、文件传输和排队所需的时间；小任务在本地运行有时更省事。平台概览

第一部分向模型 API 发送请求，本部分运行自己的计算和训练程序，两者的资源与费用分别管理。107 平台账号能申请哪些资源、使用多久，以及是否计费，以当前账号说明为准。

2.1.2 本地、登录节点与计算节点



关系示意，节点与界面以平台预演为准

- **Windows 本地**：保存代码和取回的日志，本地 agent 在这里读取、修改文件。
- **登录节点**：整理文件、编辑脚本、配置环境、提交任务。训练通过资源申请进入计算节点。
- **计算节点**：调度系统分配 CPU/GPU 后，真正运行任务。
- **共享工作目录**：让提交入口与计算任务访问代码、数据和结果；实际共享范围由平台确认。

Slurm 是任务调度系统，根据任务申请分配资源并安排执行。提交后会得到任务 ID，但可能还需要排队；查看状态和日志才能知道是否开始运行。

Web Shell、计算节点 Shell 和 VS Code 应用终端可能位于不同节点。操作前看清讲义标注的位置：虽然浏览器开在 Windows 上，平台终端中的 Linux 命令是在远程执行的。

2.1.3 上传、提交与检查 CPU 任务

步骤 1：准备与上传代码

打开 107 平台，通过统一身份认证登录，找到“文件管理”“作业”和“登录集群”等入口。界面名称如有变化，可查阅 GUI 说明。

完整解压实验 04 材料包，进入 `code/compute-practice/`，能看到 `hello.py`、`train.py`、`configs/` 和 `scripts/`。包里不含数据、环境和训练结果。

通过文件管理，将 `compute-practice/` 及其内容上传到平台用户目录下的 `projects/`。若上传的是完整材料包，可以在文件管理界面选中压缩包、使用解压缩操作，再把其中的这个项目放到 `projects/`。新建目录、上传、复制、移动与普通项目包的压缩/解压都可以在界面完成，操作方法见文件管理说明。

后文使用 `~/projects/compute-practice`[6] 作为项目根目录，打开后应直接看到 `hello.py`、`requirements.txt`、`configs/` 和 `scripts/`，避免多套一层同名文件夹。网页中的“作业”指计算任务，与课程作业提交无关。

步骤 2：填写网页任务字段

进入“作业 → 提交作业”，把工作目录设为项目根目录。在命令区域填写：

Bash · 平台终端

```
python hello.py
```

第一次选择 **CPU 任务，GPU 数为 0**，使用当前账号允许的较小 CPU/内存配置和短时限。把标准输出、错误日志位置记下来；若填写相对日志目录，先在文件管理中创建它。

`hello.py` 只使用标准库，可以先用平台已有的 Python。若页面要求使用 `python3`，将命令改为 `python3 hello.py`。

提交后记录任务 ID，在作业列表中查看状态。开始运行后，输出应包含：

示例文本

```
hello from computing platform
```

日志还会显示解释器、当前目录和节点。程序将结果保存到 `outputs/hello/result.json`，打开并下载它，核对内容与日志是否一致。这份 hello 结果会在重跑时更新，后面的训练则为每次运行保留单独目录。

[6] Linux 路径中的 ~ 表示当前用户的主目录，`$HOME` 通常也指这个位置，与 Windows 盘符不同。

2.2 CIFAR-10 图像分类的 GPU 短训练

继续使用刚才的项目，准备 Python 环境和 CIFAR-10 数据，再提交 GPU 训练。这里只运行少量步骤，主要检查 CUDA 运算、日志和模型保存，不要求达到较高准确率。

2.2.1 分步配置项目环境

以下命令在**平台登录集群的网页终端**[7]中执行。先进入刚才的项目根目录；把第一行示例路径改为实际项目位置：

Bash · 平台终端

```
cd ~/projects/compute-practice
pwd
ls
```

`pwd` 显示当前目录，`ls` 列出文件。确认这里有 `requirements.txt`、`configs/` 和 `scripts/`，再继续安装。

步骤 1：检查已有 conda；没有时再安装

Bash · 平台终端

```
command -v conda
python --version
```

已有可用 conda 时跳到步骤 2，创建独立 Python 环境。若没有，可以在用户目录安装 Miniforge。以下是 Linux x86_64 示例，先用 `uname -m` 核对平台架构，再从学校镜像下载安装器。中科大镜像说明

Bash · 平台终端

```
uname -m
mkdir -p ~/downloads
cd ~/downloads
compute_mirror=https://mirrors.ustc.edu.cn/github-release
compute_release="$compute_mirror/conda-forge/miniforge/LatestRelease"
compute_installer=Miniforge3-26.7.2-0-Linux-x86_64.sh
wget "$compute_release/$compute_installer"
bash "$compute_installer" -b -p "$HOME/miniforge3"
source "$HOME/miniforge3/etc/profile.d/conda.sh"
cd ~/projects/compute-practice
```

安装器名称按 2026-10-11 镜像目录写出；目录更新后选择对应架构的安装器。已有 `~/miniforge3` 时不要直接重复安装；检查现有环境或选择新的安装位置。这里没有改写全局软件源，`source` 为当前终端加载 conda。

[7]选中终端输出后，右键选择“复制”；粘贴时在终端输入区域按 `Ctrl+Shift+V`。`Ctrl+C` 通常会中断当前程序。多行命令先看清再粘贴，详见平台终端说明。

步骤 2：创建独立 Python 环境

conda 管理 Python 解释器和依赖。为训练项目创建单独环境，避免影响课程基础环境：

Bash · 平台终端

```
conda create --prefix "$HOME/.conda/envs/compute-resnet" \
  --override-channels \
  -c https://mirrors.ustc.edu.cn/anaconda/cloud/conda-forge \
  python=3.11 pip -y
conda activate "$HOME/.conda/envs/compute-resnet"
which python
python --version
```

`which python` 应指向 `compute-resnet/bin/python`。路径已存在时先确认是不是本项目环境，不要为了重装删除已有环境。

步骤 3：选择 PyTorch 构建并安装

本例固定使用 `torch 2.7.1 + torchvision 0.22.1`，安装组合见 PyTorch 官方历史版本页。CUDA 构建需与计算节点的 GPU 和驱动兼容，不能因为登录节点没有 GPU 就安装 CPU 包。驱动由平台维护者管理。

下面以 `cu128` 为候选示例，可参考 PyTorch 2.7 的 Blackwell 支持说明。先按平台建议确认构建，再在已激活的环境中安装：

Bash · 平台终端

```
python -m pip install torch==2.7.1 torchvision==0.22.1 \
  --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements.txt \
  --index-url https://mirrors.ustc.edu.cn/pypi/web/simple
python - <<'PY'
import torch, torchvision
print(torch.__version__, torchvision.__version__, torch.version.cuda)
PY
```

第一条命令从指定索引安装 PyTorch，第二条从学校 PyPI 镜像安装其余依赖。若下载失败，保留完整错误，查询平台推荐的镜像或离线包方案；上述下载入口尚未完成纯国内网络预演。

导入成功说明依赖可以加载，GPU 运算还要在分配到 GPU 的任务中检查。登录节点上的 `torch.cuda.is_available()` 为 False，不一定是环境安装失败。

步骤 4：使用环境配置脚本

熟悉上述步骤后，可以用配套脚本创建另一个新环境：

Bash · 平台终端

```
bash scripts/setup-env.sh cu128 "$HOME/.conda/envs/compute-resnet-next"
```

脚本接受 `cpu`、`cu126`、`cu128`，在指定位置创建环境，不覆盖已有环境。使用 GPU 构建前仍需确认兼容性。后面的提交脚本直接调用环境内 Python 的完整路径，不依赖网页终端是否激活了 conda。

2.2.2 准备 CIFAR-10 与短训练配置

CIFAR-10 包含十类 32×32 彩色图像。配套程序使用 ResNet-18，从随机初始化开始训练，并调整了第一层和池化以适应小图像。训练代码已提供，按下文准备数据即可。数据说明、ResNet-18 模型说明

步骤 1：下载与准备数据

首次仍需要完整 CIFAR-10 Python 原始包，约 163 MiB；少样本参数只减少训练用量。可以从原始数据页下载。网络慢时优先在本地下载后上传，训练脚本不在线下载数据。

在 **Windows 本地 PowerShell**, 进入解压后的 `compute-practice/` 项目根目录, 下载数据包:

PowerShell • Windows

```
Invoke-WebRequest `
  -Uri 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz' `
  -OutFile '.\cifar-10-python.tar.gz'
```

通过文件管理把压缩包上传到**平台项目根目录**。CIFAR-10 数据使用配套脚本校验并解包；在平台已配置的终端中执行:

Bash • 平台终端

```
python prepare_data.py --archive cifar-10-python.tar.gz --data-dir data
python prepare_data.py --data-dir data --check-only
```

成功时显示 `verified CIFAR-10`。脚本只提取预期的数据文件，并检查其完整性，不改写已有数据目录。校验失败时，先检查压缩包是否完整，再用新目录重新解包。

项目目录如下:

示例文本

```
compute-practice/
  hello.py                  # CPU 入门
  train.py                  # 主线与选读共用训练入口
  prepare_data.py            # 原始包校验和解包
  inspect_run.py             # 读取结果和模型
  configs/smoke.json         # 256 训练样本、8 步的短训练配置
  configs/train.json         # 较大训练候选配置
  scripts/                  # 环境与任务脚本
  data/cifar-10-batches-py/ # 已解包数据
  logs/                     # Slurm 日志，提交前创建
  outputs/                  # 按运行区分结果
```

步骤 2：读懂短训练配置

`configs/smoke.json` 使用 256 个训练样本、128 个验证样本、batch 32、最多 8 次参数更新。样本数决定看多少数据，`max_steps` 限制整个任务的更新次数；`0` 表示由 epoch[8] 决定、不额外限制。

[8] batch 是一次参数更新使用的样本数，一个 epoch 通常表示将训练集遍历一遍。本例还使用 max_steps 限制更新次数，因此可以提前结束这一轮。

程序将官方训练集固定划为 45000 个训练样本和 5000 个验证样本，短训练只取其中一小部分，官方测试集不参与。附录中的较大训练使用同一程序、种子和划分，通过配置文件增加训练量；实际耗时需在平台预跑后确定。

2.2.3 用 sbatch 提交并检查训练

步骤 1：查看资源申请字段

打开 `scripts/smoke.sbatch`，对应查看这些字段：

网页里的含义	脚本字段	示例
分区 / 资源分组	`--partition`	`Students`
资源规则	`--qos`	`qos_stu_default`
GPU 数量或类型	`--gres`	`gpu:1`
CPU 核心数	`--cpus-per-task`	`2`
CPU 主内存	`--mem`	`8G`
最长运行时间	`--time`	`00:15:00`
输出和错误日志	`--output` / `--error`	`logs/%x-%j.out/.err`

示例值需按账号权限调整，尤其是分区、QOS 和 GPU 申请写法。`#SBATCH` 行由 Slurm 解析，其中的 shell 变量不会自动展开；`%x` 和 `%j` 分别替换为作业名与任务 ID。sbatch 参数说明

步骤 2：提交并观察

在**平台提交终端，项目根目录**执行：

Bash · 平台终端

```bash
export COMPUTE_ENV_PREFIX="$HOME/.conda/envs/compute-resnet"
bash scripts/submit.sh smoke
squeue -u "$USER"
```

`submit.sh` 会先创建日志目录，再从项目根目录提交。训练调用 `COMPUTE_ENV_PREFIX/bin/python`；环境位置与示例不同时，修改这个变量。在交互终端激活 conda，不会自动替批处理选择解释器。

若账号要求指定 GPU 型号或使用其他 QOS，先查询允许的值，再修改脚本或向 `submit.sh` 传入 sbatch 参数。

记录任务 ID。下面以 `12345` 作格式示例，换成自己的 ID：

Bash · 平台终端

```bash
scontrol show job 12345
tail -n 50 logs/compute-smoke-12345.out
tail -n 50 logs/compute-smoke-12345.err
```

开始训练后，日志应显示 `device=cuda`、GPU 名称、训练步骤和保存位置。程序要求 CUDA 可用，未分配 GPU 或环境不支持时会报错，不自动改用 CPU。

任务结束后若 squeue 看不到它，继续在网页历史记录查看结果；若平台支持 accounting，也可以查：

Bash · 平台终端

```
sacct -j 12345 --format=JobID,State,ExitCode,Elapsed
```

结合任务状态、退出码、日志和输出文件判断运行结果。排队时查看等待原因，已有任务或网页应用也可能占用资源。取消任务可用 scancel 12345，把示例 ID 换成自己要取消的任务。

步骤 3：检查训练结果

默认目录为 outputs/smoke-12345/，包含：

- run.json：设备、环境版本、配置、步骤数、完成状态，以及第一层权重是否更新。
- metrics.csv：训练及验证指标、对应步骤和计算时间。
- model.pt：本次保存的模型参数。
- loss.png：结果曲线；一次短训练可能只有一个记录点。

在同一项目环境中读取：

Bash · 平台终端

```
"$COMPUTE_ENV_PREFIX/bin/python" inspect_run.py \
  outputs/smoke-12345 --require-cuda
```

脚本检查任务是否完成、记录的设备是否为 CUDA、模型能否读取，以及指标与步骤数是否对应。parameter_changed 表示第一层权重发生了更新，不代表模型已有较高准确率。检查通过后，再下载本次结果。

2.3 结果下载与问题排查

2.3.1 下载结果、复用项目与释放资源

在文件管理中进入 outputs/，将本次结果目录压缩后下载；再进入 logs/，下载对应的 .out 和 .err。大量小文件先打包会更方便。若希望将结果与日志合成一个包，可以先用文件管理将所需文件复制到新归档目录后压缩，或在**平台项目根目录**执行下面的示例，任务 ID 换成自己的：

Bash · 平台终端

```
tar -czf smoke-12345-result.tar.gz \
  outputs/smoke-12345 \
  logs/compute-smoke-12345.out logs/compute-smoke-12345.err
```

在 Windows 本地解压后，可以查看 run.json、CSV 和图片。用 inspect_run.py 读模型还需要本地安装兼容依赖，配置方法见附录 A。结果自行保留，无需提交。

批处理任务结束后会释放分配的资源。使用过临时交互会话时，用 exit 退出；使用过网页应用时，在平台确认应用已停止。关闭浏览器不一定会停止远程应用，具体操作见平台任务说明。

以后可复用已有环境和数据，新的训练使用新输出目录，并保存配置与环境版本。增加训练量的练习见附录 A。

2.3.2 演示：本地 Agent 辅助排错（可选）

配套脚本提供了一个故意使用错误数据路径的任务，用来演示怎样借助本地 Agent 排错。这是选做内容，正常练习无需制造错误。

如果刚才已关闭平台终端，重新打开登录/提交终端，进入同一 `compute-practice/` 项目根目录，并按短 GPU 训练的提交步骤重新设置 `COMPUTE_ENV_PREFIX`，指向已核对的项目环境。随后在这个平台终端提交演示任务：

Bash · 平台终端

```
bash scripts/submit.sh path-error
```

错误任务让 `scripts/run-job.sh` 的 `path-error` 分支使用 `data-does-not-exist`。环境和 GPU 检查通过后，训练程序应报数据目录不存在。记下错误任务 ID，下载该任务的 `.out`、`.err`，同时在项目目录记录：

Bash · 平台终端

```
pwd
ls data/cifar-10-batches-py
```

把日志放入 **Windows 本地项目**的一个排错记录目录，让 agent 同时看到代码与日志。可以这样请求：

示例文本

```
这是平台任务的数据路径错误演示，请读取提交脚本和我下载的日志。
命令在平台执行，本地目录只是用于修改项目。
平台工作目录和实际数据目录见我提供的 pwd、ls 输出。
请解释失败位置，并只修正 path-error 分支的数据路径。
保留 GPU 要求和资源参数，给出修改差异及平台重跑步骤。
你未在平台执行的部分请标注待验证。
```

确认数据位于项目的 `data/` 后，应让 `path-error` 分支使用这个目录。此错误与 GPU 数量和依赖安装无关。查看 Agent 的修改差异，再将修订后的 `scripts/run-job.sh` 上传到平台演示副本。

重新提交演示任务，记录新的任务 ID，并检查对应的日志、CUDA 记录和模型文件。只有重跑成功，才能确认修改在平台上有效。最后按结果下载说明保存日志和结果，并确认任务与临时应用已结束。

2.3.3 平台常见问题速查

现象	先检查什么
任务一直排队	等待原因、账号权限、已有运行任务和应用、申请资源是否过大
找不到 Python	`COMPUTE_ENV_PREFIX` 和其中的 `bin/python`；交互终端激活不会自动传给批处理
找不到 CIFAR-10	作业工作目录、`--data-dir`、目录结构和完整性校验
CUDA 不可用	是否实际分配 GPU、wheel 的 CUDA 构建、节点 GPU/驱动；保留错误再排查
提交内存超限	当前允许申请范围；与运行时 OOM 分开判断
`CUDA out of memory`	显存需求，先调整 batch 等计算配置；`--mem` 不是显存开关
没有日志	任务是否启动、提交前日志目录是否存在、日志路径与作业目录
输出目录已存在	选择新的运行目录，保留旧结果

向 AI 或助教求助时提供操作位置、任务 ID、脚本、日志、资源参数及必要目录信息。平台 FAQ、Slurm 速查

平台示例尚未完成普通学生账号的 GPU/Slurm 全流程预演，环境和资源申请需按当前账号核对。

附录 A：图像分类项目迁移与较大训练（选读）

本附录使用同一个图像分类项目，练习先在本地运行，再迁移到平台。代码已提供，本地 Agent 协助检查环境、路径和提交脚本；资源需求与运行时间通过平台预跑确定。

A.1 本地短测与迁移检查

步骤 1：在 Windows 本地做短检查

已有兼容环境时可以直接使用。需要新建环境时，在 Windows PowerShell 进入 `compute-practice/` 项目根目录，执行下面的命令。环境放在项目外的 `.venvs/`，不改动课程基础环境：

PowerShell · Windows

```
python -m venv '..\.venvs\compute-cpu'
$computePython = '..\.venvs\compute-cpu\Scripts\python.exe'
& $computePython -m pip install torch==2.7.1 torchvision==0.22.1 `
  --index-url https://download.pytorch.org/whl/cpu
& $computePython -m pip install -r requirements.txt
```

也可以使用独立 conda 环境，但要确认后续命令调用的是其中的解释器。本地这里只测试 CPU 运行，GPU 仍需到平台验证。

在本地项目中，按准备 CIFAR-10 中的方式下载并解包数据：

PowerShell · Windows

```
python prepare_data.py --archive '.\cifar-10-python.tar.gz' --data-dir '.\data'
& $computePython train.py --config configs/smoke.json `
  --device cpu --data-dir data --output outputs/local-check --max-steps 2
& $computePython inspect_run.py outputs/local-check
```

确认 `run.json`、指标、图片和模型可以读取。程序不覆盖已有结果，重跑时换一个输出目录。练习不使用 CIFAR-10 官方测试集调参。

步骤 2：准备平台配置

将平台环境路径、分区/QOS、GPU 申请写法，以及允许申请的 CPU、主内存和时限告诉 Agent，让它检查 `configs/train.json`、`scripts/train.sbatch` 和项目目录。示例请求如下：

平台可选择 RTX 5090 或 A100 等 GPU，选择时先看模型与 batch 的显存需求，再结合可用资源和预跑速度决定。CPU 核心数、主内存和时限也可按需要调整：数据加载慢时检查 CPU/worker 设置，显存不足时调整 batch 或选择显存合适的卡；先完成单卡预跑，再考虑是否需要更多资源。

示例文本

```
请把这个本地已短测的 CIFAR-10 / ResNet-18 项目准备为平台单卡任务。
平台环境和资源规则以我提供的实际信息为准。
请检查数据路径、解释器、训练配置、日志和输出目录。
先解释修改理由，再修改必要的配置和提交脚本。
保持固定数据划分与种子，不自动扩大到多卡。
给出我在平台执行的命令和应检查的结果。
```

Windows 环境目录不用上传到 Linux；在平台按版本说明建立环境。只上传代码、配置和所需数据，已有平台数据可复用。

A.2 扩大训练与取回结果

步骤 1：增加训练量

`configs/train.json` 提供一组较大训练配置：45000 个训练样本、5000 个验证样本、batch 128、10 个 epoch。`max_steps=0` 表示不额外限制更新次数。这组配置尚未在平台预跑，不能确定是否能在示例时限内完成。

可以先复制为自己的配置，依次增加样本数、训练步数和 epoch。先观察 CPU workers、显存和训练速度，再决定是否改 batch、资源或申请更长时限。

在已核对环境与资源的**平台项目根目录**：

Bash · 平台终端

```
export COMPUTE_ENV_PREFIX="$HOME/.conda/envs/compute-resnet"
bash scripts/submit.sh train
```

`scripts/train.sbatch` 示例申请单卡、4 CPU、8G 主内存和 1 小时。按账号权限和预跑结果调整，避免申请用不到的资源。若因用户资源上限排队，也要检查已有任务和应用是否仍在运行。

步骤 2：下载与读取结果

记录任务 ID，查看对应日志和 `outputs/train-任务 ID/`。检查任务是否完成、使用的 CUDA 设备，以及模型和指标文件。

下载结果后，在本地环境读取，例如任务 ID 为 `12346`：

PowerShell · Windows

```
& $computePython inspect_run.py outputs/train-12346 --require-cuda
```

检查脚本将模型加载到本地 CPU；`--require-cuda` 检查保存的记录是否来自 CUDA 训练，本地电脑无需 GPU。

图表中的训练损失来自训练时的批次，验证损失来自每轮末尾的模型，可以用来观察训练变化。运行更多步骤也不一定获得较高准确率，还需结合数据和参数分析。

A.3 观察计算速度与资源需求

CPU/GPU 比较使用同一脚本、相同数据规模、batch 和步骤数，固定并记录线程/worker 设置。CPU 只测有限步数，不必先把较大任务完整跑完。

`run.json` 保存训练时间、处理图片数、每秒处理图片数和 GPU 张量内存峰值。计时包含数据加载与参数更新，并在 CUDA 运行时进行同步。短任务容易受启动开销影响，比较时要说明使用的配置。PyTorch 计时指南

比较完整任务的耗时时，还要计入下载、上传、环境配置和排队。PyTorch 记录的显存峰值不包含设备全部用量，也不同于 CPU 主内存，需结合平台资源统计判断。

保存本次代码版本、配置、环境信息和结果，便于以后重跑或迁移其他项目。

附录 B：配置管理与工具扩展（选读）

以下工具供需要管理多套配置或连接额外服务时参考，本实验的练习无需另行安装。

B.1 用管理器维护多套配置

CC Switch 提供图形界面，用于添加、切换 API 提供方，并管理 MCP、Skills、提示词和用量。项目采用 MIT 许可证。支持的提供方和配置方式见发布说明，切换后重启客户端并测试一次调用。

Cockpit Tools 偏向多平台账号、配额、多实例与供应商管理，适合已有多个工具和项目的用户。源码公开，默认许可为 CC BY-NC-SA 4.0，带非商业限制。

这些管理器负责修改配置或提供本地路由，模型服务和计费仍由实际 API 提供方负责。切换前备份配置，并确认 Key 与服务地址对应；尽量避免多个管理器同时修改同一份配置。

B.2 Skills：保存重复任务的方法

Skill 通常包含某类任务的说明、脚本和资源。例如将 Python 项目的检查步骤写成一份技能，方便以后重复使用。客户端决定怎样加载和触发技能，以及它能调用哪些工具。

需要重复处理同类任务时，可以参考 Codex Skills 官方说明，了解文件结构和触发方式。

B.3 MCP：接入工具与数据

MCP 是 Model Context Protocol，规定客户端怎样连接外部工具和资源服务。例如通过 MCP 服务提供资料搜索或数据库查询工具，让 Agent 调用。使用前需确认客户端支持 MCP、模型能调用工具，以及服务所需的权限。

需要连接额外数据源或工具时，可参考 MCP 官方入门和 Codex MCP 配置说明。本实验使用客户端已有的文件和终端工具。

B.4 Codex CLI 补充入口

如需命令行补充入口，在已经安装 Node.js 的 Windows 环境中按 Codex CLI 官方说明运行：

PowerShell • Windows

```
npm install -g @openai/codex
codex
```

CLI 与桌面客户端共享用户配置。在练习目录启动 `codex`，并确认模型和工作目录。上述安装命令需要访问 npm 下载源，遇到下载错误时与 API 调用问题分开排查。Codex 可直接在本地使用；DSH 的安装与配置见附录 C。

附录 C：DSH 介绍与配置（选读）

DSH（DeepSeek Harness）是 DeepSeek 开发的开源 Harness。它组织模型服务、会话、文件与终端工具，让模型根据工具返回的结果继续推进任务。在本实验中，可以用它读取决策树项目、修改 T1、调用课程 Python 运行自查，再根据报错修正代码。官方项目

它采用插件架构，文件操作、终端、任务组织与其他扩展能力可以按需组合。除编程外，也可用于整理资料、运行数据处理脚本、生成文档；具体能力取决于所接模型、已启用的工具和权限。首次练习先使用客户端已有的文件与终端工具，再按需要了解插件、Skills 等扩展。功能介绍

DSH 提供工作的运行环境，模型仍通过所选服务完成推理。换用 DSH 后，API 地址、Key 和模型 ID 仍要与提供方对应；用量和费用也记在实际模型服务账户中。下面以桌面界面接入学校 API 为例。

C.1 选择发行版，安装并启动

可以直接在 Windows 本机安装并使用，先准备一份公开练习项目的副本。虚拟机[9]是需要额外隔离时的可选方式，不作为本教程的配置前提。官方仍将 DSH 标为尚未经过安全审计的开发者预览软件；审批和工作区权限有助于限制操作，但不能保证完整隔离。使用前保留项目备份，并了解官方安全说明。

先选择发行版

- DeepSeek 官方桌面版由官网提供 Windows 64 位安装入口。
- 社区 DSH Desktop 是 dataelement 维护的独立桌面应用，负责启动本机 Harness，并提供工作区选择、模型设置和更新等桌面功能。跟随学校图文指南时，采用这一发行版，优先选择其下载页中的稳定发行版。

两者有共同的上游框架，安装包、界面和数据目录可能不同，记录自己使用的版本即可。以下配置字段与学校指南中的社区桌面版对应。

安装并打开客户端

1. 在所选发行版的下载页选择适合本机架构的 Windows 安装包，按提示安装。
2. 从桌面或开始菜单打开客户端，等待本机运行环境启动。社区 DSH Desktop 的安装包已经包含所需运行环境，无需为这条桌面路线另装 Node.js。
3. 若社区版首次出现模型服务商向导，使用学校 Key 的同学选择“稍后配置”，再按 C.2 添加学校提供方；默认 DeepSeek 提供方连接的是 DeepSeek 官网。

Codex 的 PowerShell 接入脚本修改的是 Codex 配置。DSH 按自己的模型设置界面配置，不会因已配置 Codex 就自动接入同一服务。

[9]虚拟机通过虚拟化软件，在当前电脑上运行一套独立的操作系统，例如在 Windows 中再运行 Windows 或 Linux。快照可保存某一时刻的状态，方便试用后恢复；共享给虚拟机的宿主机目录仍可能被访问或修改。

C.2 连接学校 API

在“设置 → 模型 → 添加自定义提供方”填写：

字段	学校服务示例
提供方 ID	`ustc`
显示名称	中科大公共平台
Base URL / API 地址	`https://api.llm.ustc.edu.cn/v1`
API 协议	`openai-completions`，即 Chat Completions
API Key	你申请的学校实验 Key
模型 ID	从当前项目授权模型列表复制准确 ID，保留后缀

点击“获取可用模型”，添加项目已授权的模型并保存；如果列表获取失败，也可以手动添加准确的模型 ID。然后回到会话中选中它。Key 输入框直接填写学校 Key，与学校网关配合使用。使用 DeepSeek 官方 Key 时，按客户端的 DeepSeek 提供方说明配置，费用记在对应官方账户。DSH 模型服务配置

C.3 查看权限，验证真实调用

首次练习保留审批提示和受限工作区模式，先使用默认插件与本机连接。使用官方模型服务时，查看“设置 → 常规”中的会话日志上传选项，按实际资料选择；学校自定义服务与官方服务的路由分别核对。会话日志说明

点击“选择工作区”，添加并选中决策树练习副本；再选择刚配置的模型，新建会话。工作区选择影响当前任务的文件位置，模型选择决定请求发给哪个服务。先发送一次仅回答文字的请求：

示例文本

```
请用三句话解释模型与 Harness 的区别。
这次只回答文字，不读取或修改文件。
```

收到回答后，到服务平台查看项目用量。再按第 1.3 章先读代码，再补全 T1 并运行自查。将课程 Python 的完整路径告诉 DSH，查看工具调用中读取的文件、执行的命令、退出状态和实际输出；最终应看到代码差异与 `5/5 passed`，仅有文字回答不能说明检查已经运行。

C.4 保存结果，结束试用

保存需要的学习结果，关闭客户端并确认运行进程结束。仅为本次试用创建的临时 Key 可在对应服务平台撤销；保留常用 Key 的原有安排。需要重做时，可以重新复制练习项目，或按附录 D 恢复已保存的代码版本；选择虚拟机运行的同学也可使用自己的快照。

遇到版本或权限问题时，可以先用 Codex 完成本实验的练习。

附录 D：用 Git 保存版本，降低试错成本（选读）

写程序时，经常会遇到“刚才还能运行，改了几处却不知道哪里出了问题”的情况。Git 可以把一个可用版本保存为提交，随后查看改动，或在尝试失败后取回先前的代码。让 Agent 修改项目前先留下一份这样的快照，试错就容易得多。

这里先介绍本地版本管理。保存提交、查看差异和恢复文件都可以离线完成，无需注册账号；GitHub 等网站提供远端仓库，便于共享这些提交。

D.1 在改动前保存一个版本

Windows 可从 Git 官方安装页安装 Git，安装后重新打开 PowerShell。把决策树练习复制到课程仓库以外的位置，例如自己的文档目录，再进入这份副本的根目录。下面的 `git init` 用于为副本建立本地仓库；已有 Git 仓库的项目可以直接查看状态。

在 **Windows 本地 PowerShell · 决策树练习副本根目录**执行。名字和邮箱改成自己希望记在提交中的信息，这两项只设置当前仓库：

PowerShell · Windows

```
git --version
git init
git config user.name "你的名字"
git config user.email "你的邮箱"
git add decision_tree.py checks.py README.md AGENTS.md
git commit -m "保存决策树初始代码"
```

`git add` 选择本次要保存的文件，放入暂存区；`git commit` 将暂存内容保存为一个版本。提交说明帮助你以后辨认它。这里保存的是初始骨架，T1–T7 仍未完成；也可以在完成 T1 并通过检查后，再保存一个可用版本。

Git 仓库的记录放在项目的 `.git` 目录中。上面的提交只包含列出的四份文件，不包含 Python 环境、训练数据或运行结果；新文件若未加入提交，也不会出现在旧快照里。

D.2 看清改动，再保存下一版

让 Agent 修改 `decision_tree.py` 后，先查看哪些文件变化了，以及具体改了什么：

PowerShell · Windows

```
git status
git diff -- decision_tree.py
```

`git diff` 默认显示尚未暂存的差异。运行程序和对应阶段的自查，确认修改可用，再暂存并检查本次要保存的内容：

PowerShell · Windows

```
git add decision_tree.py
git diff --cached -- decision_tree.py
git commit -m "实现信息熵并通过检查"
git log --oneline -5
```

`--cached` 查看暂存区与上次提交之间的差异；`git log` 列出最近的提交。这样，初始骨架、T1 完成版和后续修改都有各自的记录，不必靠“最终版 2”“最终版 3”区分文件。查看差异

D.3 尝试失败后，取回之前的代码

如果这次试改还没有提交，并且确定要放弃 `decision_tree.py` 中所有未提交的变化，可以先查看差异，另存其中仍需保留的内容，再执行：

PowerShell • Windows

```
git restore --source=HEAD --staged --worktree -- decision_tree.py
git status
```

`HEAD` 表示当前分支最新的提交。这条命令将指定文件的暂存内容和工作区内容都恢复为该版本，其他文件不受影响。文件中尚未提交的修改会被覆盖，因此要先看清差异再决定。恢复文件

如果错误修改已经提交，先确认工作区没有其他未提交的改动，并用 `git log --oneline -5` 看清最近的记录。**仅当最新一次提交就是要撤回的错误修改**时，可以执行：

PowerShell • Windows

```
git revert --no-edit HEAD
git log --oneline -5
```

`git revert` 新建一次提交，抵消那次修改；原来的记录仍然保留。如果出现合并冲突，先查看提示和差异，必要时让 Agent 帮助解释；不准备继续处理时，可用 `git revert --abort` 取消这次撤回操作。撤回提交

与 Agent 协作时，可以要求它先查看 `git status` 和 `git diff`，说明已有改动，再只修改本轮涉及的文件。遇到问题也先比较差异，确定要取回哪个文件、哪个版本；不要仅说“全部回滚”，以免把别的工作一起撤掉。

后续需要共享代码时，`push` 将本地提交上传到远端仓库；PR（Pull Request）用于提出、讨论和审阅合并请求，`merge` 才将分支改动合到一起。本地保存版本已经可以帮助试错，这些协作操作可按项目需要再学。Git 中文入门书、GitHub PR 简介